Credential Routes Filter: Enhancement

Module: Master → Supplier → Credential Routes Filter **Change type:** Enhancement **Date:** 13 Oct 2026 **Repo:** odeysys (branch base `main`)

> **Purpose**
> This is a technical analysis and implementation guide for the four requirements in the user story. It records how the feature works in the code today, the gap for each requirement, a proposed design with the files it touches, the questions the business must answer before development starts, and a test plan. Every file and line reference was checked against the repository on the date above.

1. Explain it like I'm 5
2. How the feature works today
3. Requirement 1: route limit 50 → 150
4. Requirement 2: airline in the supplier request
5. Requirement 3: configuration model
6. Requirement 4: deprecation flags
7. Open questions for the business
8. Problems already in the code
9. Test plan
10. Rollout and estimate
11. File index

1. Explain it like I'm 5

The toy shop story

Picture a big toy shop (our travel system). Kids (agents) ask: "*I want a plane ride from Dubai to London!*"

The shop doesn't own any planes. It calls **helpers** on the phone (suppliers such as Galileo, Sabre, Amadeus and Travelport NDC). Each helper has several **phone lines** (credentials / PCCs).

The **Credential Routes Filter** is a rule card stuck next to each phone line. It says things like:

- "*Use this phone line only for Dubai → London trips.*" (a whitelist of routes)
- "*...and only for Emirates planes.*" (the airline)

What changes:

1. **Bigger rule card.** Some helpers' cards hold only 50 trips and others hold 150. Now every card holds 150.
2. **Ask for the right thing up front.** Today we phone the helper and say "*Show me every plane to London,*" then throw away everything that isn't Emirates. It's like ordering the whole menu and eating one dish. From now on we say "*Show me only Emirates planes to London.*" It's faster, we waste less, and we get more Emirates choices back.
3. **One airline per card.** Each rule card names one airline, one helper, one phone line, and the trips it covers.
4. **Retiring the old notebook.** An older notebook, the "Preferred Airline" screen, did a similar job. We keep it visible (switch ON) while the grown-ups copy its notes onto the new cards. When they finish, they flip the switch OFF and the old notebook disappears.

2. How the feature works today

2.1 Data model

Table	Purpose	Key points
`TT_TS_SUPPLIER_CREDENTIAL_ROUTES` V20260421110000	Route rules per credential	`CREDENTIAL_ID`, `ORIGIN`, `DESTINATION` (3-letter code or `ALL`), `IS_BLACKLIST`. The blacklist flag is **copied onto every route row**, and code reads it from `routes.get(0)`.
`TT_TS_SUPPLIER_CREDENTIAL_AIRLINES` V20260629101403	Airlines attached to a credential's rule	`CREDENTIAL_ID` and `AIRLINE_ID` (FK to `TT_TS_AIRLINES.CONTENT_ID`). Unique on (credential, airline), so a credential can have **many** airlines. There is no link between an airline and a specific route.
`TT_TS_PREFER_AIRLINE` (legacy)	"Preferred Airline" master, the screens being deprecated	`SUPPLIER_NAME`, `PCC_ID`, `AIRLINE_ID`, `STATUS`. It has **no routes**. This is the **only** source the GDS connectors use today to add carriers to the supplier request.

2.2 Admin API (odeysysadmin-tt)

`SupplierCredentialRouteRuleControllerV2` serves `/credential-routes-filter`: GET `/{credentialId}`, POST (add), PATCH (edit, implemented as delete-all then re-insert), DELETE, GET list (paged), GET `/view/{id}`. The business logic is in `SupplierCredentialRouteRuleApplicationServiceImpl.upsert()`:

- Normalises routes, maps `-1` to `ALL`, rejects ALL→ALL, and removes duplicates.
- **Route limit:** 150 when the request's `supplierType == GDS`, otherwise 50 (lines 44–51 and 183–191).
- Only one rule set is allowed per credential. ADD fails with "This credential already has configured routes."
- Writes an audit entry (`NDCAuditModuleEnum.Credential_Routes_Filter`).

2.3 Flight search pipeline (where the rules are applied)

1. Build request
FlightRestService loads the **Preferred Airline master** into `prefAirlinesFromMaster`, keyed `Supplier_PccId`

→

2. Pick credentials
FlightSearchFiltrationService:61 calls `SupplierCredentialRouteFilterServiceImpl`, which drops credentials whose **route** rules don't match

→

3. Call suppliers
Galileo, Sabre and Amadeus add carriers from the **master** only. Travelport NDC sends **no** carriers

→

4. Post-filter
FlightWSService:1169 calls `SupplierCredentialAirlineFilterServiceImpl`, which removes flights whose **airline** doesn't match

> **The key gap behind Requirement 2**
>
> Airlines configured in Credential Routes Filter are applied only in **step 4**, after the supplier has already answered. The supplier returns every airline on the route, and we discard most of them. Suppliers cap result counts, so the configured airline can be partly or fully crowded out of the response. That loses fares and wastes response size and time.

2.4 How the connectors add carriers today

Supplier	Where	Element sent	Source today
Galileo GDS	`GalileoUtil.java:324` (+531, 8319)	`<PermittedCarriers><Carrier Code=".."/>`	Preferred Airline master
Sabre ATPCO GDS	`SabreRequestUtil.java:5701, 6135, 7415`	`TPA_Extensions.IncludeVendorPref[{Code}]`	Preferred Airline master
Amadeus GDS	`AmadeusRequestUtil.java:7247, 7594, 7694, 7952`	`companyIdentity / carrierQualifier M / carrierId`	Preferred Airline master
Travelport NDC	`ndcintegration/refactor/operations/NDCSearchOperation.java:71–97`	**None.** `AirSearchRequestDto` has no carrier field	—

The three GDS connectors all use one helper, `FlightUtil.createPreferAirlineListBasedOnSupplierAndPcc()` at `FlightUtil.java:6775`. That gives us **one place** to plug in the new source.

3. Requirement 1: route limit 50 → 150 Small

Current: `SupplierCredentialRouteRuleApplicationServiceImpl.java:44-51, 183-191` branches on the **client-sent** `supplierType`.

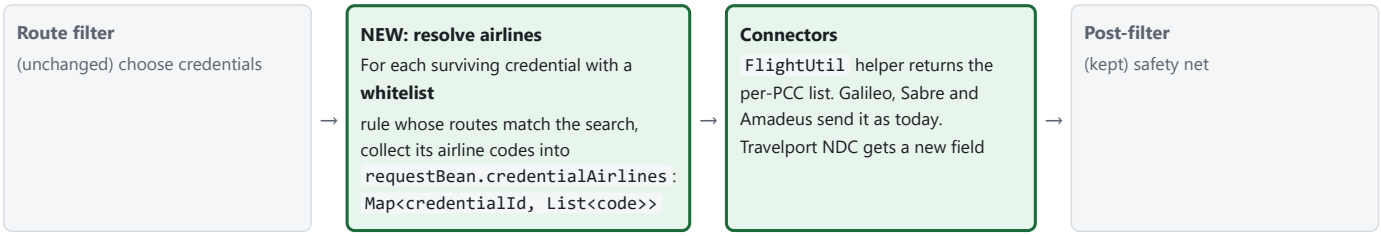
Change: Replace the two constants with a single `MAX_ROUTE_RULES_PER_BATCH = 150` and remove the branch.

```
private static final int MAX_ROUTE_RULES_PER_BATCH = 150;
private static final String MAX_ROUTE_PER_BATCH_ERROR = "Maximum 150 routes are allowed in one batch.";
...
if (normalizedRoutes.size() > MAX_ROUTE_RULES_PER_BATCH) {
    throw new BadRequestExc(MAX_ROUTE_PER_BATCH_ERROR);
}
```

- Removing the branch also fixes two existing defects. A request without `supplierType` currently throws an NPE (HTTP 500), because the field has no `@NotNull`. And the limit was trusted from the client, so a caller could send `GDS` for an LCC credential.
- `supplierType` then has no use in the request DTO. Keep the field (ignored) so existing front-end calls don't break, and mark it deprecated.
- **Front-end:** check that the admin UI doesn't hard-code 50, for example in an "Add route" counter or a validation message.
- **Performance:** one rule set can now reach 150 rows on LCC. Matching is in memory over the list. That's fine at this size, so no index changes are needed.

4. Requirement 2: airline in the supplier request Large

4.1 Proposed design



1. **Extract shared route matching first.** `RoutePattern`, `normalizePattern`, `normalizeAirportOrAll` and `extractRequestRoutes` are copy-pasted in both filter services. Move them into one `CredentialRouteMatcher` in common-odeysys before adding a third consumer, so all three always agree on what "matches" means.
2. **New resolver** (common-odeysys, called from `FlightSearchFiltrationService` right after line 61). It reuses the route/airline maps already loaded there (`fetchRouteRuleEntitiesByCredentialIds` and `fetchAirlineRuleEntitiesByCredentialIds`), so it adds **no extra DB round trip**. Batch-resolve the airline codes with `findAirlineCodeByAirlineIds` (one query, not one per credential as the post-filter does today).
3. **Carry the result per credential.** Add `Map<Integer, List<String>> credentialAirlines` to `FlightSearchRequestBean` (ttesb-tt). It must be keyed by credential, because one request bean fans out to many suppliers. A flat list would leak one credential's airline into another credential's request.
4. **One connector change point.** Extend `FlightUtil.createPreferAirlineListBasedOnSupplierAndPcc()` to return the credential-filter airlines for that PCC. While the deprecation flag is ON, merge them with the master list (see Q4). The Galileo, Sabre and Amadeus call sites then need almost no changes. Also relax the outer guard `getPrefAirlinesFromMaster()!=null && !isEmpty()` at each call site. Otherwise an empty master map skips the new airlines.
5. **Travelport NDC.** Add a carrier field to `models/primarytypes/AirSearchRequestDto.java`, for example `preferredAirlines` to mirror `NDCAPI… Filters`. Set it in `NDCSearchOperation.prepareFlightSearchRequest()` around line 85. **Dependency:** the downstream `/api/FlightSearch/Search` service must accept and honour this field. Confirm with the NDC integration owners before committing a date.
6. **Keep the post-response filter.** Suppliers don't always enforce carrier filters strictly (codeshares, operating vs. marketing carrier), so step 4 stays as a guarantee.

4.2 Rules to respect

- **Whitelist only.** Injecting the airline as a "permitted" carrier is correct only in whitelist mode. In blacklist mode it would do the opposite of the rule. Blacklist either sends nothing (default) or uses each supplier's "prohibited" element (Galileo `ProhibitedCarriers`, Sabre `ExcludeVendorPref`, Amadeus qualifier `X`). See Q2.
- Keep the existing exclusions: `EXCLUDED_AIRLINES` and `validateRestrictedAirlines(supplierConfig)`.
- If the agent also picked preferred airlines in the search widget (`getPrefAirlines()`), the request should use the **intersection**. If the intersection is empty, skip that credential instead of sending an unfiltered search. See Q3.

- Inject only for the four suppliers in scope. LCC connectors don't change.

5. Requirement 3: configuration model Medium, depends on Q1

	Today	Required
Airlines per record	0..n (`airlineIds` list, optional)	**Exactly 1**
Supplier	Derived from credential	1 (still derived; UI cascade supplier → credential)
Credential	1. Only one rule set per credential	1
Routes	0..n (empty list silently saves nothing)	**1..n**, max 150

Two ways to implement it, depending on what the business means:

Option A: validation only (one record per credential)

- Require `airlineIds.size() == 1` and `routes` not empty in `upsert()`. Better still, replace `airlineIds` with `@NotNull Integer airlineId` in a new DTO version.
- Effect: each credential serves exactly one airline. No schema change.
- Data: existing credentials with 2+ airlines need a cleanup script or a manual review before deploy.

Option B: new record table (several records per credential)

- New parent table `TT_TS_SUPPLIER_CREDENTIAL_ROUTE_CONFIG (ID, CREDENTIAL_ID, AIRLINE_ID, IS_BLACKLIST, audit cols)`, unique on (credential, airline). Routes gain a `CONFIG_ID` FK, and `IS_BLACKLIST` moves to the parent where it belongs.
- This allows "PCC X: Emirates on DXB→LHR, and Qatar on DOH→JFK", with each airline tied to its own routes. That matches the story's "record" wording and is what migrating Preferred Airline data (per PCC and airline) will need.
- Cost: API and DTO changes (id-based endpoints instead of credential-based), a data migration from the two current tables, and an update to the search-time filters. Both filters then match routes per *(credential, airline)* instead of per credential.

> **Recommendation**
> Choose **Option B** if any credential has to serve more than one airline, which is very likely for GDS PCCs. Otherwise Option A is a one-day change. This is the first decision to get from the business (Q1).

> **Migration blocker**
> Preferred Airline rows mean "always send airline X on PCC Y", with **no routes**. The new model requires at least one route, and the existing business rule forbids ALL→ALL ("You can select All in origin or destination, but not both"). As things stand, the business **cannot** express a legacy Preferred Airline row in the new screen. Either allow ALL→ALL for this case or agree that every migrated row gets real routes. Decide this **before** the deprecation plan starts (Q5).

6. Requirement 4: deprecation flags (admin) Medium

Use the existing settings mechanism. Don't invent a new one: rows in `TT_TS_SETTINGS`, read through `SystemSettingService.isActionEnabled(key)`, where `"1"` means ON. Values are cached in Redis.

```
INSERT INTO `wonderdb`.`TT_TS_SETTINGS`
(`ACTION_KEY`, `ACTION_VALUE`, `ACTION_CLASS`, `ACTION_STATUS`, `DATA_TYPE`, `DESCRIPTION`) VALUES
('PREFER_AIRLINE_SCREENS_ENABLED', '1', '0', '0', 'boolean', 'Show Preferred Airline list/view screens (deprecated by Credential Routes
Filter)'),
('PREFER_AIRLINE_EDIT_ENABLED',    '1', '0', '0', 'boolean', 'Allow add/edit/status-change on Preferred Airline (deprecated)');
-- ROLLBACK: DELETE FROM `wonderdb`.`TT_TS_SETTINGS` WHERE ACTION_KEY IN ('PREFER_AIRLINE_SCREENS_ENABLED','PREFER_AIRLINE_EDIT_ENABLED');
```

- **Why two flags.** The business can first freeze edits (stop new legacy data while migrating) and later hide the screens completely. Adjust if the business wants exactly one.
- **Hide in the UI:** skip the Preferred Airline menu item when the flag is OFF (menu rendering from `TT_TS_MENUS`).
- **Enforce on the server:** hiding a menu is not security. Guard every endpoint in `PreferAirlineControllerV2` (`Master2/preferAirline`): add, update and status endpoints check the edit flag, and search and view check the screens flag. Return 403 with a clear message.
- **Cache:** after the admin flips a flag, evict the Redis key (or say how long the TTL is) so the change shows up right away.
- **Audit** flag changes if the settings screen doesn't already.

> **Hiding the screens doesn't switch off the data**
>
> `FETCH_PREFER_AIRLINE` loads every `STATUS=1` row into each search, whatever the flags say. If the business turns the screens OFF while old rows are still active, those rules stay **live and invisible**: nobody can see them or edit them. Mitigation: when `PREFER_AIRLINE_SCREENS_ENABLED` is OFF, `FlightRestManager.getPreferAirlineFromMaster()` returns an empty map. Otherwise, a runbook step must deactivate all rows before the switch. See Q4.

Deprecation path: Deploy (flags ON) → business recreates rules in Credential Routes Filter → verify search results per PCC → edit flag OFF (freeze) → screens flag OFF (hidden, legacy data ignored) → a later release removes the code and drops `TT_TS_PREFER_AIRLINE`.

7. Open questions for the business

#	Question	Why it matters	Recommended default
Q1	Can one credential have several records (one per airline), or is it one airline per credential?	Option A vs. Option B (section 5). This changes the schema and API.	Several records (Option B)
Q2	For **blacklist** rules, should the airline be sent to the supplier as "excluded"?	Sending it as "permitted" would invert the rule.	Don't send it. Rely on the post-filter
Q3	If the agent picks a preferred airline that differs from the configured one, what should happen?	Empty intersection	Skip that credential
Q4	While flags are ON, do Preferred Airline *and* the new config both drive the request (merged)? When OFF, is legacy data ignored?	Avoids double rules and invisible live rules	Merge while ON. Ignore when OFF
Q5	How do we migrate route-less Preferred Airline rows? Do we allow ALL→ALL?	Blocks migration (section 5)	Allow ALL→ALL only when an airline is set
Q6	Does the Travelport NDC search service support a carrier filter?	External dependency for Requirement 2	Confirm with the NDC team
Q7	Is the story's "One airline" final, given the current UI allows several?	Existing multi-airline data needs cleanup	Report affected credentials before deploy

8. Problems already in the code

I found these while reading the feature. They are already there today, and the enhancement is a good time to fix them because the same code is being changed.

#	Where	Issue	Impact
1	`SupplierCredentialRouteRuleApplicationServiceImpl.java:183`	`request.getSupplierType().equals(..)` with no `@NotNull` on the field	HTTP 500 when the field is missing. Fixed by Requirement 1
2	`SupplierCredentialAirlineFilterServiceImpl.java:69`	Wraps any exception in `RuntimeException`, while the route filter fails open	One DB hiccup fails the **entire** search
3	`SupplierCredentialAirlineFilterServiceImpl.java:130, 189, 220`	`Integer.parseInt(flight.getSearchPCCId())` with no guard	NFE/NPE fails the search if a supplier leaves it empty
4	Both filter services	Route-matching logic is duplicated	Logic can drift. Extract it (section 4.1, step 1)
5	`…AirlineFilterServiceImpl.buildAirlineCodesMap`	One `findAirlineCodeByAirlineIds` query per credential	N+1 queries on every search. Batch them
6	Routes table	`IS_BLACKLIST` stored per route row and read from `get(0)`	Mixed flags would give undefined behaviour. Fixed by Option B
7	Admin vs. search	"ALL" sentinel is `-1` in admin and `-11` in search filters	Harmless today (stored as `ALL`) but confusing
8	`V20260501233137`	Inserts `INT_CODE 19`, but the rollback comment deletes `17`	The rollback would delete the wrong enum row
9	`upsert()` EDIT path	Delete-all then insert, with an empty route list returning early	An edit with no routes silently wipes the config and skips the audit log

9. Test plan

Unit

- Limit: 150 routes pass and 151 fail for both LCC and GDS. A missing `supplierType` no longer returns 500.
- Model: 0 airlines, 2 airlines and 0 routes are each rejected with a 400 and a clear message.
- `CredentialRouteMatcher`: OW / RT (return leg) / MC, ALL on origin, ALL on destination, lower case, non-3-letter codes.
- Resolver: whitelist + matching route gives the airline. Non-matching route gives nothing. Blacklist gives nothing. Per-credential isolation (credential A's airline never appears for credential B).
- `FlightUtil` helper: merges master and credential airlines while the flag is ON, returns credential airlines only when OFF, and works when the master map is empty.

Integration (request payload snapshots)

- Galileo: `PermittedCarriers` contains the configured airline. Sabre: `IncludeVendorPref`. Amadeus: `companyIdentity M`, for OW, RT and MC builders (all call sites listed in 2.4).
- Travelport NDC: the new field is present in the `/api/FlightSearch/Search` body.
- LCC suppliers: payload unchanged (regression).

Flags

- ON/ON: menu visible and CRUD works. Edit OFF: add/update/status return 403 and view works. Screens OFF: menu hidden, all endpoints 403, and legacy data no longer injected.
- A flag flip takes effect without a restart (cache eviction).

UAT

- For a sample PCC per supplier: before and after, count of results for the configured airline on a configured route. Expect the same or more results and a faster response.

10. Rollout and estimate

Work item	Module(s)	Size
Route limit + validation fixes (Req 1, Option A part of Req 3)	odeysysadmin-tt	0.5–1 d
Extract `CredentialRouteMatcher` + resolver + request bean field	common-odeysys, ttesb-tt	2 d
GDS connectors via `FlightUtil` + guard changes	ttesb-tt	1.5–2 d
Travelport NDC field (after downstream confirmation)	ttesb-tt (+ NDC service)	1 d + external
Option B schema + API + data migration (if chosen)	all	4–5 d
Deprecation flags: settings, menu, controller guards, master bypass	common-odeysys, odeysysadmin-tt	1.5 d
Tests + QA support	—	2–3 d

These are rough engineering estimates for one developer who knows the codebase, and they exclude front-end work. Build order is common-odeysys → ttesb-tt → odeysysadmin-tt / odeysysportal-tt.

> **Release safety**
> Flags default ON, so the old behaviour is preserved on deploy. Carrier injection is additive and the post-filter stays, so the worst case is an unchanged result set. Ship each supplier's injection behind its own log line (credential, airline codes sent) so production can be checked per PCC.

11. File index

Area	File
Admin API	`odeysysadmin-tt/…/v2/controller/supplierCredentialController/SupplierCredentialRouteRuleControllerV2.java`
Admin logic + limits	`odeysysadmin-tt/…/v2/application/service/supplierCredential/SupplierCredentialRouteRuleApplicationServiceImpl.java`
Request DTO	`odeysysadmin-tt/…/v2/model/supplierCredentialFilter/SupplierCredentialRouteRuleModuleReqDTO.java`
Entities / DAO	`common-odeysys/…/ts/supplier_credential_route_rules/{entity,dao}/*`
Pre-search route filter	`common-odeysys/…/odeysysportal/flight/service/SupplierCredentialRouteFilterServiceImpl.java`
Post-search airline filter	`common-odeysys/…/odeysysportal/flight/service/SupplierCredentialAirlineFilterServiceImpl.java`
Filter orchestration	`common-odeysys/…/odeysysportal/flight/service/FlightSearchFiltrationService.java` (line 61) `common-odeysys/…/ws/rest/air/service/FlightWSService.java` (line 1169)

Legacy Preferred Airline	`common-odeysys/…/ts/preferAirline/model/PreferAirlineModel.java`
	`common-odeysys/…/odeysysportal/flight/manager/FlightRestManager.java` (line 133)
	`odeysysadmin-tt/…/v2/controller/preferAirline/PreferAirlineControllerV2.java`

Request bean	`ttesb-tt/…/flight/bean/flightsearch/FlightSearchRequestBean.java`
Carrier helper	`ttesb-tt/…/services/util/FlightUtil.java` (line 6775)
Galileo / Sabre / Amadeus	`ttesb-tt/…/connector/util/{GalileoUtil, SabreRequestUtil, AmadeusRequestUtil}.java`
Travelport NDC	`ttesb-tt/…/connector/ndcintegration/refactor/operations/NDCSearchOperation.java` `ttesb-tt/src/main/java/models/primarytypes/AirSearchRequestDto.java`
Settings flags	`common-odeysys/…/systemconfiguration/service/SystemSettingService.java` (`TT_TS_SETTINGS`)
Migrations	`V20260421110000` (routes), `V20260629101403` (airlines), `V20260423164758` (menu 4003007), `V20260501233137` (audit enum)